

Mpho Brian Mhlabi

Systems Development NQF5

Advanced Programming II Assignment.

DUE DATE : 22 SEPTEMBER 2026

LECTURER : MR OWAMI

Introduction

The desktop application called Data validation system developed with the use of Java Swing in order to capture a student's / person's information and validate the information using a given criteria. The information captured is the users' name and surname, ID number or student ID, Date of birth, Gender and email address as well as their phone number.

The main purpose of this system is to make sure that incorrect and /or incomplete information is not accepted into the system. With the use of if statements value length checks and information validation, and provides this feedback to the user.

System Design

The interfaces' components use conditional statements and labels next to the text fields ensuring that a user can visibly identify which information to correct / change. Instead of normal text fields, gender is selected with the use of radio buttons. At the bottom of the form, there are 'Save', 'Clear', and 'Exit' buttons which serve the function of validation, clearing the text fields and starting fresh or exiting and closing the system/form.

The design consists of a simple and straight-forward layout which is easy to use and understand, with a large enough font for visually impaired people, and a validation feedback feature .

Fields Used

The following are the fields that were used and what they were meant to capture:

Full name : a JTextField was used to capture a user's full name.

Student/ID Number : a JTextField was used to capture the user's ID or Student number (13 digit number) .

Date of Birth : a JTextField was used to capture a user's Date of birth (yyyy-mm-dd format).

Contact Number : a JTextField was used to capture a user's Phone number (10 digit contact Number).

Email Address : a JTextField was used to capture a user's Email address .

Gender : a JTextField was used to capture a user's Gender (Male or female).

Validation Checks

The validation feature is accessed/enabled when the user clicks the "Save" button, and with the use of conditional statements, the input is logically checked and feedback is provided to the user.

Name field

The name field first checks whether or not the field is empty, then check that only letters are used. The field also checks if the name is more than 2 characters and less than 30.

Student / ID number

The ID field checks for whether or not the field is empty and then checks that the user has only entered digits that are no smaller or larger than an index of 13.

Date of Birth

The Date of Birth field is used to check for empty values and converts the present value into a date format of YYYY-MM-DD. Any invalid dates are rejected.

Name Validation

The name field first checks whether it is empty. It then checks that only letters and spaces are used. Finally, it checks that the name is between 2 and 30 characters. This prevents blank, numeric or excessively long names from being accepted.

```
String name = nameTxt.getText();

    if (name.isEmpty()) {

        nameVal.setText("Field is empty");

        nameVal.setForeground(Color.red);

    }

    else if(name.matches("[a-zA-Z]+") && name.length() >=3){

        nameVal.setText("VALID");

        nameVal.setForeground(Color.green);

        IbIResultName.setText(name);

    }

    else if(name.length() < 2 || name.length() > 30){

        nameVal.setText("Enter a valid full name");

        nameVal.setForeground(Color.red);

    }

}
```

ID/Student Number Validation

The ID field checks for an empty value, checks that only digits are entered, and then checks that the value contains exactly 13 digits.

```
String id = idTxt.getText();

    if (id.isEmpty()){

        idVal.setText("Field is empty");

        idVal.setForeground(Color.red);

    }

}
```

```

else if(!id.matches("\\d+")){
    idVal.setText("Invalid");
    idVal.setForeground(Color.red);
}
else if(id.length() != 13)
{
    idVal.setText("13 No. ID");
    idVal.setForeground(Color.red);
}
else{
    idVal.setText("Valid");
    idVal.setForeground(Color.green);
    IbIResultID.setText(id);
}
}

```

Date of Birth Validation

The date of birth field checks for an empty value and attempts to convert the input into a `LocalDate` using the `yyyy-MM-dd` pattern. Invalid dates are rejected, and future dates are not accepted.

```

String age = birthTxt.getText();

//int ages = Integer.parseInt(birthTxt);

if (age.isEmpty()) {
    birthVal.setText("Field is empty");
    birthVal.setForeground(Color.red);
}

else if(!age.matches("\\d+")){
    birthVal.setText("Use Numbers");
    birthVal.setForeground(Color.red);
}

else{
    birthVal.setText("Valid");
    birthVal.setForeground(Color.green);
    IbIResultAge.setText (age);
}

```

```
}
```

Gender Validation

The gender field uses two radio buttons. Selecting one option deselects the other. When SAVE is pressed, the system checks that at least one gender option has been selected.

```
String gender = maleRadio.getText();

if (gender.isEmpty()) {
    genderVal.setText("Field is empty");
    genderVal.setForeground(Color.red);
}

else if (maleRadio.isSelected()) {
    genderVal.setText("Male Selected");
    genderVal.setForeground(Color.green);
    IbIResultGender.setText(gender);
}

else if (femaleRadio.isSelected()) {
    genderVal.setText("Female Selected");
    genderVal.setForeground(Color.green);
    IbIResultGender.setText(gender);
}

else if (femaleRadio.isSelected() && maleRadio.isSelected()) {
    genderVal.setText("Select 1 Gender");
    genderVal.setForeground(Color.red);
}
```

Contact Number Validation

The contact number must not be empty, must contain digits only, and must contain exactly 10 digits.

```
String number = numberTxt.getText();

    if(number.isEmpty()){

        numberVal.setText("Field is empty ");
        numberVal.setForeground(Color.red);

    }

    else if(!number.matches("\\d+")){

        numberVal.setText("Enter a number");
        numberVal.setForeground(Color.red);

    }

    else if(number.length() != 10){

        numberVal.setText("Enter 10 digit number");
        numberVal.setForeground(Color.red);

    }

    else{

        numberVal.setText("Valid");
        numberVal.setForeground(Color.green);

        IbIResultNumber.setText(number);

    }

}
```

Email Validation

The email field checks that a value is present, uses a basic email pattern, and does not exceed 100 characters. The regular expression is used as the format check.

```
String email = emailTxt.getText();

    if(email.isEmpty()){

        addressVal.setText("Field is empty");
        addressVal.setForeground(Color.red);

    }

    else if(email.contains("@gmail.com")){

        addressVal.setText("Valid");
        addressVal.setForeground(Color.green);

        IbIResultEmail.setText(email);

    }

    else{
```

```
        addressVal.setText("invalid");  
        addressVal.setForeground(Color.red);  
    }  
}
```

Testing

Testing was designed to demonstrate that invalid input is rejected and that valid input is accepted. The evidence should show the value entered and the feedback produced by the application. Screenshots should be taken directly from the running application.

The systems fields are empty and the Save button is clicked in order to initiate the validation process. Once the user clicks "save", the systems text fields will all display a JLabel that indicates "Field is empty".

The screenshot shows a Java Swing application window titled "Static Data". The window has a teal background and a dark teal sidebar on the right. The main area contains six text input fields arranged in two columns. The left column fields are labeled "Full Name", "Date Of Birth", and "Contact Number". The right column fields are labeled "ID/Student number", "Gender" (with radio buttons for "Male" and "Female"), and "Email Address". Each field is currently empty and has a red label "Field is empty" below it. At the bottom, there are three buttons: "SAVE", "CLEAR", and "EXIT".

The fields are then filled with dummy text that when processed through the validation system :

Name : field is detecting that there is less than 3 characters, enabling the JLabel to display invalid name in red.

The screenshot shows a Java Swing application window with a teal background. The window has a title bar with standard OS controls. The main content area is divided into two sections. The left section, titled 'Static Data' with a magnifying glass icon, contains a form with six input fields: 'Full Name' (containing 'M'), 'ID/Student number' (empty), 'Age' (empty), 'Gender' (with radio buttons for 'Male' and 'Female'), 'Contact Number' (empty), and 'Email Address' (empty). Red text labels indicate validation errors: 'Enter a valid full name' under the Full Name field, and 'Field is empty' under the ID/Student number, Age, Contact Number, and Email Address fields. At the bottom of this section are three buttons: 'SAVE', 'CLEAR', and 'EXIT'. The right section, titled 'RESULTS', contains six empty input fields corresponding to the form fields on the left, each preceded by a label: 'Full Name:', 'ID/Student ID:', 'Age:', 'Gender:', 'Contact Number:', and 'Email Address:'.

ID/Student ID : field detects whether the value within the field is a text or integer. If the detects presence of a text, a message will be displayed informing the user to ensure the value is numbers only and should be a valid ID with 13 numbers/digits.

The screenshot shows a web application window titled "Static Data". The main form area has a teal background and contains several input fields with labels and validation messages:

- Full Name**: An empty text input field with a red message "Field is empty" below it.
- ID/Student number**: A text input field containing "mpho" with a red message "Invalid" below it.
- Age**: An empty text input field with a red message "Field is empty" below it.
- Gender**: Two radio button options, "Male" and "Female", both of which are unselected.
- Contact Number**: An empty text input field with a red message "Field is empty" below it.
- Email Address**: An empty text input field with a red message "Field is empty" below it.

At the bottom of the form area are three buttons: "SAVE", "CLEAR", and "EXIT".

On the right side of the window is a dark teal sidebar titled "RESULTS". It contains five empty text input fields, each preceded by a label:

- Full Name:
- ID/Student ID:
- Age:
- Gender:
- Contact Number:
- Email Address:

The screenshot shows a web application window titled "Static Data". The main form area has a teal background. It contains six input fields arranged in two columns. The left column has fields for "Full Name" (containing "Mpho"), "Age" (containing "18"), and "Contact Number" (containing "012345678"). The right column has fields for "ID/Student number" (containing "1234567890123"), "Gender" (with radio buttons for "Male" and "Female", where "Male" is selected), and "Email Address" (empty). Below the "Contact Number" field is a red error message: "Enter 10 digit number". Below the "Email Address" field is a red error message: "Field is empty". At the bottom of the form are three buttons: "SAVE", "CLEAR", and "EXIT". To the right of the form is a "RESULTS" section with a dark teal background, displaying the values entered in the form fields.

Field	Value
Full Name	Mpho
ID/Student ID	1234567890123
Age	18
Gender	Male
Contact Number	
Email Address	

Phone number : the text field detects the presence of text or numbers and in an event of text filled text field, the systems outputs a message to the user instructing them to ensure the field is only filled with numbers at an index of 10.

The screenshot shows a Java Swing application window titled "Static Data". The main form area has a teal background and contains several input fields with labels and validation feedback:

- Full Name:** Input field contains "Mpho", with "VALID" feedback below it.
- ID/Student number:** Input field contains "1234567890123", with "Valid" feedback below it.
- Age:** Input field contains "18", with "Valid" feedback below it.
- Gender:** Radio buttons for "Male" (selected) and "Female". Feedback "Male Selected" is shown next to the "Male" button.
- Contact Number:** Input field contains "1234567890", with "Valid" feedback below it.
- Email Address:** Input field contains "Mpho", with "Invalid" feedback below it.

At the bottom of the form are three buttons: "SAVE", "CLEAR", and "EXIT".

On the right side of the window is a "RESULTS" panel with a dark teal background, displaying the entered data in a list:

- Full Name: Mpho
- ID/Student ID: 1234567890123
- Age: 18
- Gender: Male
- Contact Number: 1234567890
- Email Address:

Email address : the email address text field detects whether or not the user has entered a valid email or not. This procedure is possible because the text field can only accept a value that contains "@gmail.com".

The screenshot shows a web application window titled "Static Data". The main form area has a teal background and contains six input fields arranged in two columns. The left column contains "Full Name" (with value "Mpho"), "Age" (with value "18"), and "Contact Number" (with value "1234567890"). The right column contains "ID/Student number" (with value "1234567890123"), "Gender" (with "Male" selected), and "Email Address" (with value "Mpho@gmail.com"). Each input field has a green "Valid" message below it. At the bottom of the form are three buttons: "SAVE", "CLEAR", and "EXIT". To the right of the form is a "RESULTS" section with a dark teal background, which displays the same data entered in the form fields.

Field	Value	Status
Full Name	Mpho	Valid
ID/Student number	1234567890123	Valid
Age	18	Valid
Gender	Male	Male Selected
Contact Number	1234567890	Valid
Email Address	Mpho@gmail.com	Valid

In any event where the data/information that the user inputs does not meet a specific requirement or is not within the scope of criteria, then a message will be displayed to the user instructing them on what to do in order for information to be accepted.

Conclusion

The Data validation system developed has ensured that a practical experience is provided the process of using Java Swing Forms and designing them.

The projects main challenges were understanding and implementing the conditional statements that ensure validation is possible, demonstrating that a user input should be validated in stages by first checking if a value exists and whether it meets the given requirements.

Names , numbers and email addresses use regular expressions and date of birth uses validate dates. This shows the importance of testing a system , both valid and invalid input , to ensure that the system functions in every scenario.

References

https://youtu.be/xTtL8E4LzTQ?si=NuEY_4djwUN5Y2hJ

[Java Swing GUI Full Course](#) 

[Introduction to Java Swing - GeeksforGeeks](#)

Github link : [MrWick109r \(BRAVO\)](#)